
Machine Learning 1 - Assignment 1 Report

Linear and Logistic Regression; Gradient Descent

Isek George Obazi

This report summarizes implemented solutions, numerical results, and visualizations for core machine learning tasks involving regression, classification, and optimization.

Project Scope

- Linear and polynomial regression models
- Logistic regression with feature engineering and evaluation
- Gradient descent optimization on the Griewank function

1. Linear Regression

1.1 Univariate model and closed-form solution

For a single feature x and target y , the model is $f_{\theta}(x) = b + w x$ and the mean-squared error is

$$L_U(\theta) = (1/N) * \sum_{i=1}^N (b + w x^{(i)} - y^{(i)})^2$$

Differentiating with respect to b and w and setting both derivatives to zero gives the normal equations:

$$dL/db = (2/N) * \sum_i (b + w x^{(i)} - y^{(i)}) = 0$$

$$dL/dw = (2/N) * \sum_i x^{(i)} (b + w x^{(i)} - y^{(i)}) = 0$$

From the first equation,

$$b^* = y_{\text{bar}} - w^* x_{\text{bar}}$$

Substituting this into the second equation yields the closed-form slope and intercept:

$$w^* = \sum_i (x^{(i)} - x_{\text{bar}})(y^{(i)} - y_{\text{bar}}) / \sum_i (x^{(i)} - x_{\text{bar}})^2$$

$$b^* = y_{\text{bar}} - w^* x_{\text{bar}}$$

These are exactly the expressions implemented in `fit_univariate_lin_model`.

Observed variable pairs

The following three pairs show a strong linear trend, while the next three are weak or not meaningfully linear.

Pair	Pearson r	Theta = [b, w]	MSE
displacement -> weight	0.9427	[1515.7802, 7.5623]	81628.1669
cylinders -> displacement	0.9492	[-132.1990, 60.0579]	1128.2041
horsepower -> weight	0.8799	[961.1535, 19.3461]	165480.5128
acceleration -> origin	0.1686	[0.8257, 0.0488]	0.6228
model_year -> origin	0.1838	[-1.4070, 0.0396]	0.6193
weight -> model_year	-0.2919	[79.5707, -0.0013]	12.6401

Interpretation: Pearson correlation lies in $[-1, 1]$. Values near 1 indicate a strong positive linear association, values near -1 indicate a strong negative linear association, and values near 0 indicate little linear association. In the scatter plots, displacement-weight and cylinders-displacement are strongly increasing, while the weak pairs show much more scatter and near-flat fitted lines.

A very small positive slope cannot coexist with a Pearson correlation near 1 in the usual non-degenerate setting, because both quantities are driven by the same covariance term. If the fitted slope is approximately zero, then the covariance is small, so the correlation cannot be close to 1. Result comment: the three strong pairs are sensible from a physical point of view. Heavier cars usually have larger engines and more cylinders, so the fitted positive slopes and high Pearson correlations are consistent with the structure of the automobile data set. By contrast, acceleration and origin or model_year and origin behave much more like noisy associations than stable linear laws, which explains the much flatter fitted lines and the smaller absolute correlations.

Discussion: for the report, the important point is not only that the model computes a line, but that the line is useful only when the scatter pattern supports a roughly linear trend. This comparison between strong and weak pairs shows why visual inspection, correlation, and MSE should be interpreted together rather than in isolation.

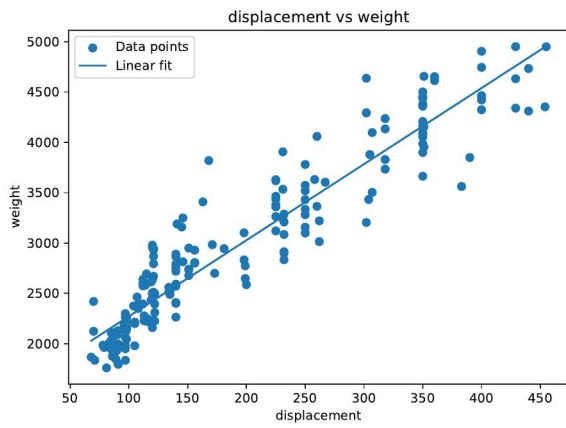


Figure 1. displacement -> weight (strong positive linear trend).

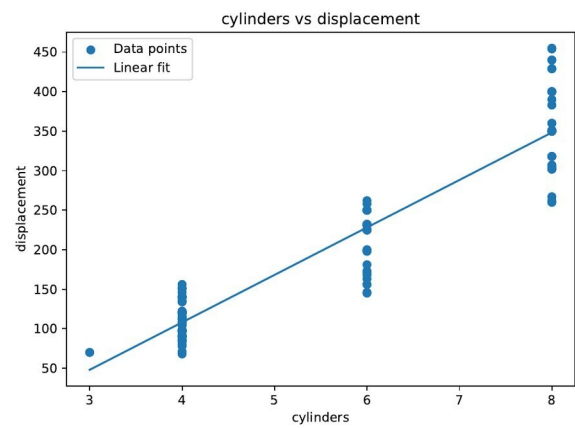


Figure 2. cylinders -> displacement (strong positive linear trend).

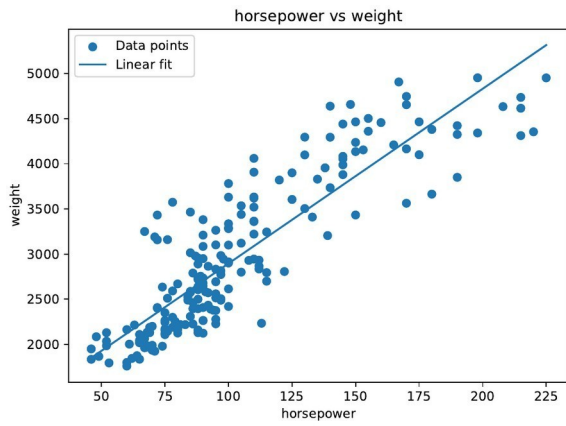


Figure 3. horsepower -> weight (strong positive linear trend).

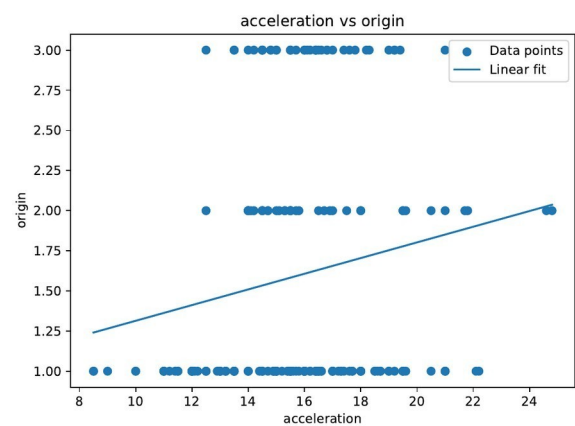


Figure 4. acceleration -> origin (weak relationship).

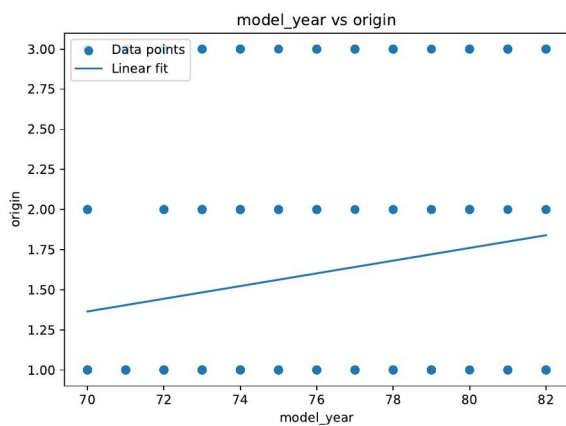


Figure 5. model_year -> origin (weak relationship).

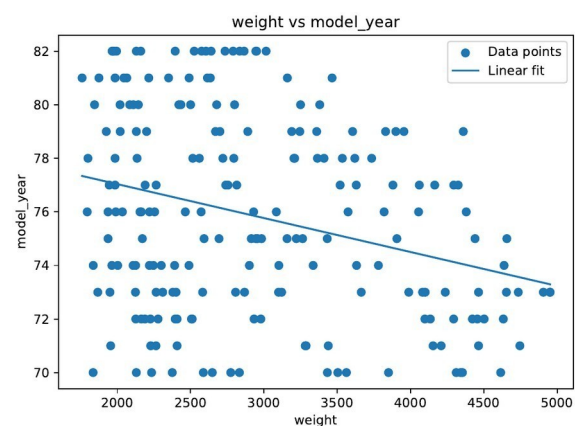


Figure 6. weight -> model_year (modest negative trend, not strongly linear).

1.2 Multiple Linear Regression

For D features, the design matrix stacks one row per sample as $[1, x_1^{(i)}, \dots, x_D^{(i)}]$. Therefore X has shape $N \times (D+1)$, θ has shape $(D+1) \times 1$, and the loss can be written as

$$L_M(\theta) = (1/N) ||X \theta - y||_2^2$$

Differentiating with respect to θ gives

$$\text{grad}_{\theta} L_M(\theta) = (2/N) X^T (X \theta - y)$$

Setting this to zero yields the normal equations $X^T X \theta = X^T y$. Using the Moore-Penrose pseudoinverse, the least-squares solution is

$$\theta^* = \text{pinv}(X) y$$

Chosen target: weight. Added predictors: displacement, mpg, and model_year. The fitted parameter vector is $\theta^* = [-171.0531, 5.8750, -38.9794, 38.4431]$ with $\text{MSE} = 57525.8353$.

Hence the fitted model is

$$\text{weight}_{\text{hat}} = -171.0531 + 5.8750 \cdot \text{displacement} - 38.9794 \cdot \text{mpg} + 38.4431 \cdot \text{model_year}$$

Compared with the univariate displacement $\rightarrow$ weight model ($\text{MSE} = 81628.1669$), the multiple model fits better ($\text{MSE} = 57525.8353$). This is expected because the univariate model is a special case of the multiple model obtained by setting the additional weights to zero. Therefore the minimum training loss of the larger model class cannot be worse.

Result comment: the reduction in MSE from 81628.1669 to 57525.8353 shows that displacement alone does not explain vehicle weight completely. Adding mpg and model_year captures extra variation that is ignored in the univariate model, so the fitted surface adapts better to the training data. The improvement is meaningful but not perfect, which suggests that weight still depends on additional unmodelled factors such as body type, manufacturer design choices, and measurement noise.

1.3 Polynomial Regression

For a polynomial of degree K , the design matrix contains the features $[1, x, x^2, \dots, x^K]$. For sample i , row i is $[1, x^{(i)}, (x^{(i)})^2, \dots, (x^{(i)})^K]$.

Using mpg as feature and weight as target, I chose $K = 2$. The fitted coefficients are $[7817.4830, -331.8099, 4.7848]$ and the MSE is 143732.7692. This is lower than the univariate linear weight $\leftarrow$ horsepower model but not lower than the strongest displacement $\rightarrow$ weight linear pair; it mainly shows that a mild curved trend can be captured when the scatter plot suggests nonlinearity.

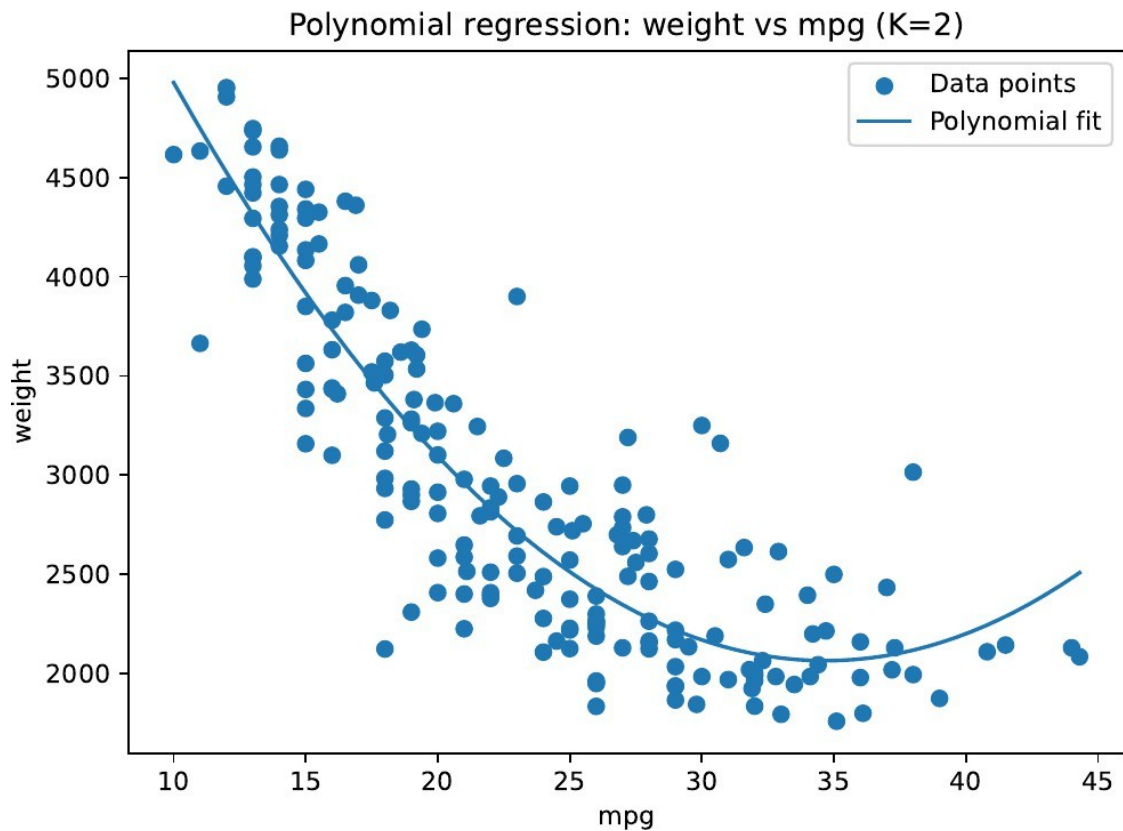


Figure 7. Polynomial regression for weight vs mpg with $K = 2$.

For x_{small_1} and y_{small_1} , the smallest degree giving essentially zero loss is $K = 4$ (observed MSE $1.227\text{e-}06$). With five distinct x -values, a degree-4 polynomial has five free coefficients, so the corresponding 5×5 Vandermonde system is square and full rank; therefore the polynomial can interpolate all five points exactly.

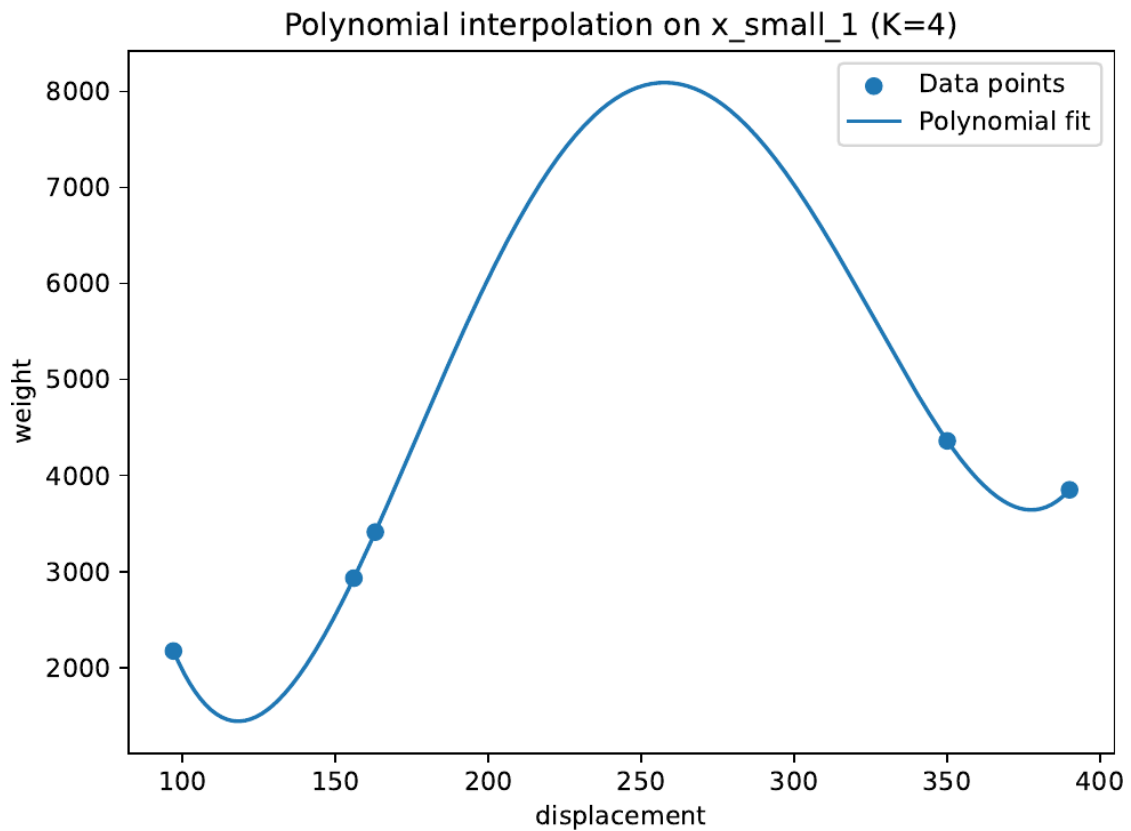


Figure 8. Exact interpolation of x_small_1 using a degree-4 polynomial.

Using the same $K = 4$ for x_small_2 and y_small_2 gives a non-zero loss (MSE 17640.0000). The reason is that $x = 250$ appears twice with different targets (3100 and 3520), so the data no longer describes a function $y = f(x)$. A polynomial assigns only one value to each x , which violates the deterministic function assumption needed for exact interpolation.

Discussion: these two small-sample experiments highlight both the power and the risk of polynomial regression. When the data truly behaves like a function with distinct x -values, a high-enough degree can interpolate perfectly. However, exact interpolation should not be confused with better generalization: the oscillatory curve in the small-set figures is a useful warning that low training loss can coexist with unstable behaviour between observed points.

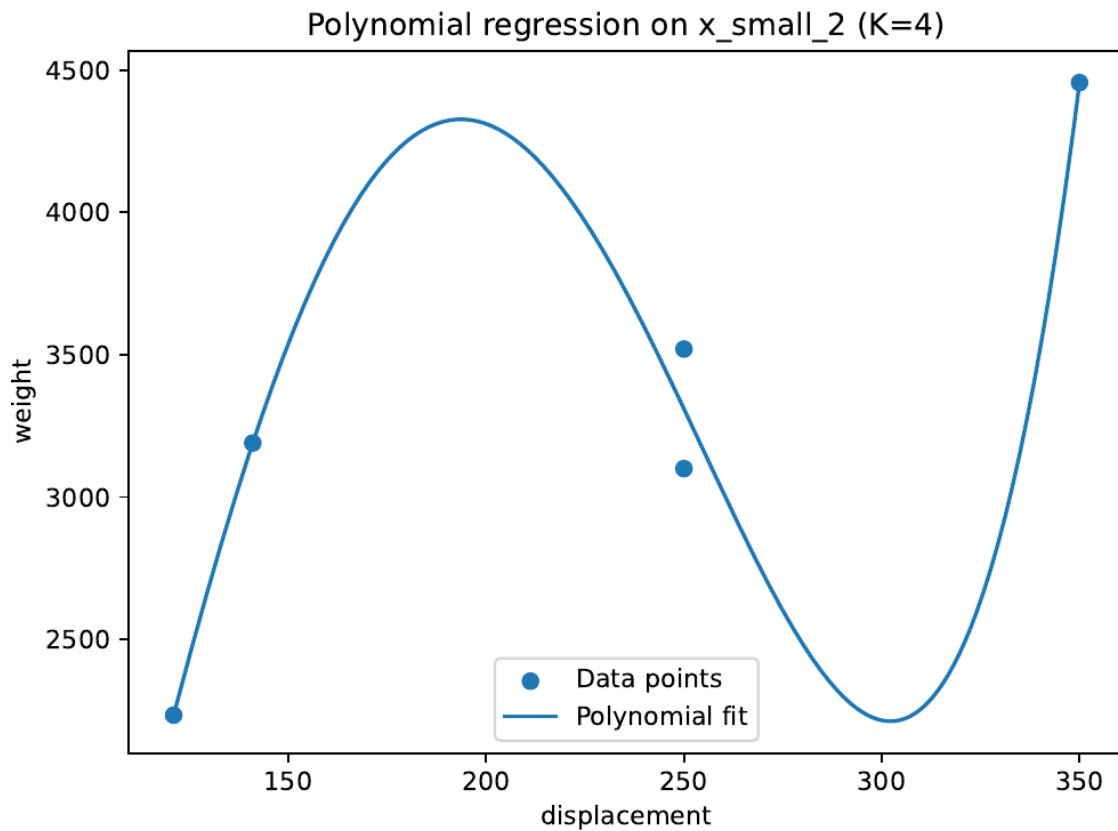


Figure 9. Degree-4 polynomial fit on x_small_2 ; exact interpolation is impossible because one input value has two different targets.

2. Logistic Regression

The submitted code uses these design matrices:

Dataset	Design matrix
1	$x_1, x_2, x_1+x_2, x_1-x_2, (x_1+x_2)^2, (x_1-x_2)^2$
2	x_1, x_2, x_1^2
3	All monomials $x_1^a x_2^b$ with total degree 1 to 5

The classifier uses `solver = lbfgs` and `max_iter = 10000`. With `lbfgs`, the default regularization is L2. I also checked other reasonable choices locally (L1 with `liblinear`, L2 with `liblinear`, `elastic-net` with `saga`, and an unregularized model). L2 with `lbfgs` matched or tied the best test accuracy on all three datasets while remaining stable, so it is a defensible final choice for this submission.

Dataset	Train acc.	Test acc.	Train CE	Test CE
1	99.23%	99.38%	0.0142	0.0267
2	100.00%	100.00%	0.0468	0.0425

3	99.06%	98.75%	0.0509	0.0679
---	--------	--------	--------	--------

Dataset 1 parameters: intercept = -59.1728; coefficients = [0.4234, 0.4322, 0.8556, -0.0088, 0.0563, -0.0677].

Dataset 2 parameters: intercept = -37.8631; coefficients = [5.2063, 0.9023, -0.1799].

Dataset 3 parameters: intercept = 0.9845; coefficients = [-3.7458, 2.9857, -0.9840, 0.7108, 0.3337, -1.6996, -1.3016, -1.3321, 1.4986, -1.9807, -0.0953, -0.7943, -0.5608, -0.4449, 2.3976, 0.1871, 1.1730, -0.3244, 0.1413, 0.0736].

The statement "100% accuracy implies zero cross-entropy loss" is false. Accuracy depends only on the predicted class label, whereas cross-entropy depends on the predicted probabilities. A classifier can classify every point correctly with probabilities such as 0.51, 0.60, or 0.90; in all these cases the accuracy is 100%, but the cross-entropy remains strictly positive. The loss becomes zero only in the limiting case where the model assigns probability 1 to the correct class for every point.

Result comment: the logistic regression results are strong across all three data sets. Dataset 2 is the easiest case because the chosen design matrix already matches the visible decision shape very well, leading to 100% accuracy on both splits. Dataset 3 is the most challenging because the class boundary is more irregular, yet the model still reaches 98.75% test accuracy, which indicates that the richer polynomial-style feature map is expressive enough without causing severe overfitting.

Discussion: the small train-test gaps are important evidence that the final parameter choice generalizes well. In this context, reporting both accuracy and cross-entropy is valuable: accuracy shows whether points are classified correctly, while cross-entropy shows how confident those classifications are. The combination gives a more informative evaluation than either metric alone.

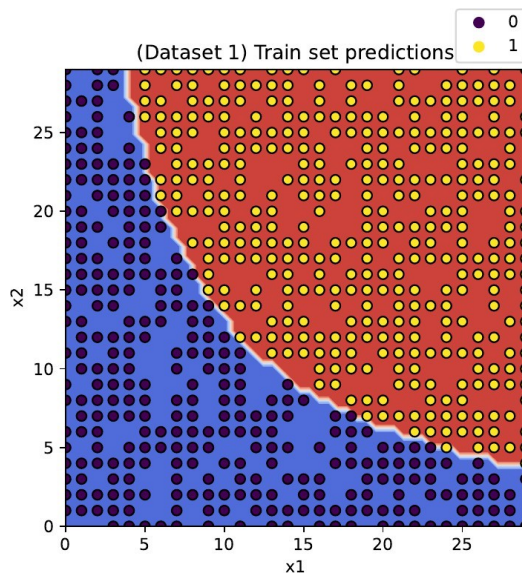


Figure 10. Logistic regression decision boundary, dataset 1 train split.

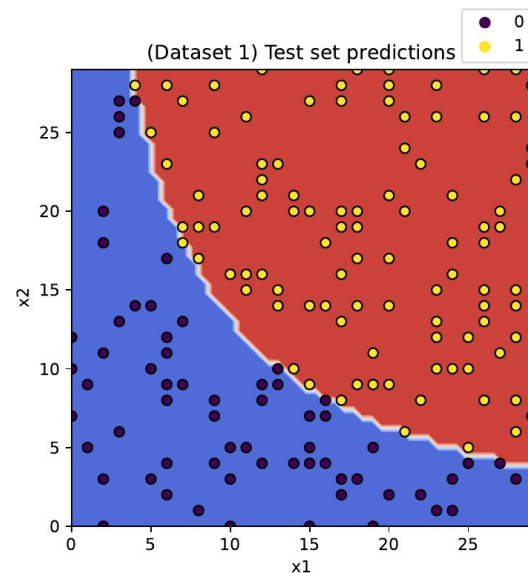


Figure 11. Logistic regression decision boundary, dataset 1 test split.

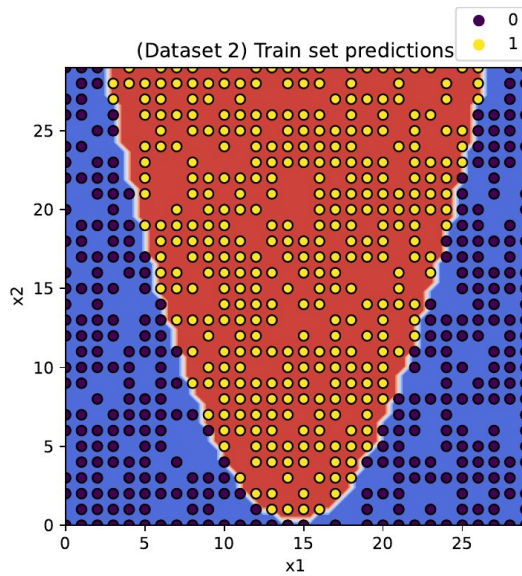


Figure 12. Logistic regression decision boundary, dataset 2 train split.

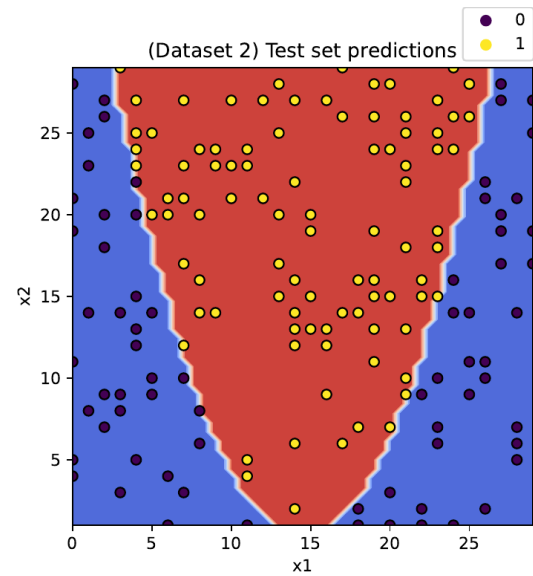


Figure 13. Logistic regression decision boundary, dataset 2 test split.

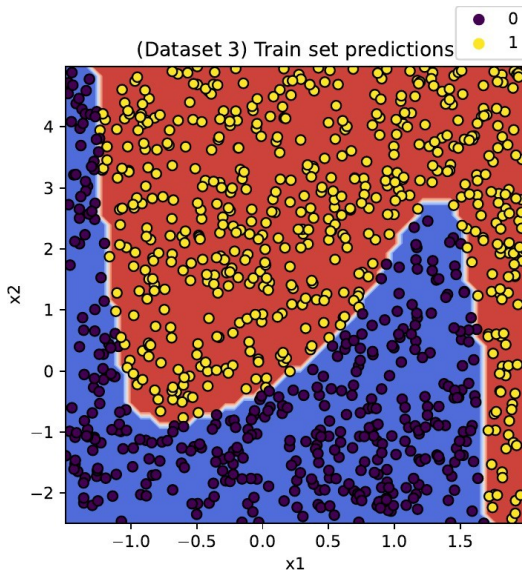


Figure 14. Logistic regression decision boundary, dataset 3 train split.

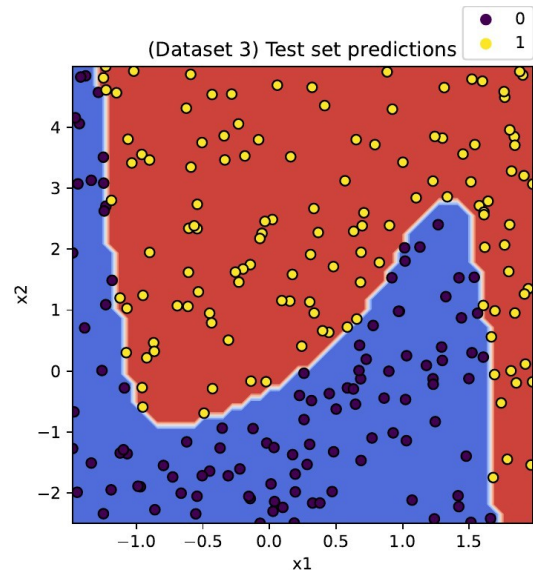


Figure 15. Logistic regression decision boundary, dataset 3 test split.

3. Gradient Descent on the Griewank Function

The objective is

$$f(x, y) = 1 + (x^2 + y^2)/4000 - \cos(x) \cos(y / \sqrt{2})$$

Gradient descent uses the update rule $x_{t+1} = x_t - \eta_t \cdot \frac{df}{dx}$ and $y_{t+1} = y_t - \eta_t \cdot \frac{df}{dy}$, with $\eta_0 = \text{learning_rate}$ and $\eta_t = \text{lr_decay} \cdot \eta_{t-1}$.

Differentiating the objective gives

$$d f / d x = x/2000 + \sin(x) \cos(y / \sqrt{2})$$

$$d f / d y = y/2000 + \cos(x) \sin(y / \sqrt{2}) / \sqrt{2}$$

For a differentiable function, the central finite-difference approximation is

$$d f / d x \approx (f(x+h, y) - f(x-h, y)) / (2h)$$

$$d f / d y \approx (f(x, y+h) - f(x, y-h)) / (2h)$$

With numpy seed 0, the sampled starting point is $(x_0, y_0) = (0.9763, 4.3038)$. The analytical gradient at this point is $[-0.82392209, 0.04104398]$ and the finite-difference approximation is $[-0.82392209, 0.04104398]$. The two values agree to many decimal places, so the numerical approximation is excellent.

Using $\text{learning_rate} = 0.1$, $\text{lr_decay} = 0.999$, and $\text{num_iters} = 500$, the algorithm converges to $(3.1400, 4.4384)$ with $f(x, y) = 0.0074$, which satisfies the brief's target $f(x, y) < 0.01$.

The function is challenging to optimize because the cosine terms create many local minima and oscillatory gradients. The optimizer can easily get trapped in a nearby basin. A decaying learning rate helps because large early steps explore the surface, while smaller later steps reduce oscillation and allow the iterate to settle. If $\text{lr_decay} = 1.0$, the step size never shrinks, so the method is more likely to overshoot and bounce around the valley instead of stabilizing near a minimum.

Result comment: the final value $f(x, y) = 0.0074$ is below the target threshold, so the gradient descent implementation is working correctly. The close match between the analytical gradient and the finite-difference approximation also gives strong evidence that the derivative formulas were implemented accurately.

Discussion: the contour figure and the objective-versus-iteration figure complement each other. The contour plot shows the actual search path in parameter space, while the decreasing objective curve confirms numerically that the method makes rapid early progress and then settles as the learning rate decays. Together, the two plots support the conclusion that the chosen settings are stable and effective.

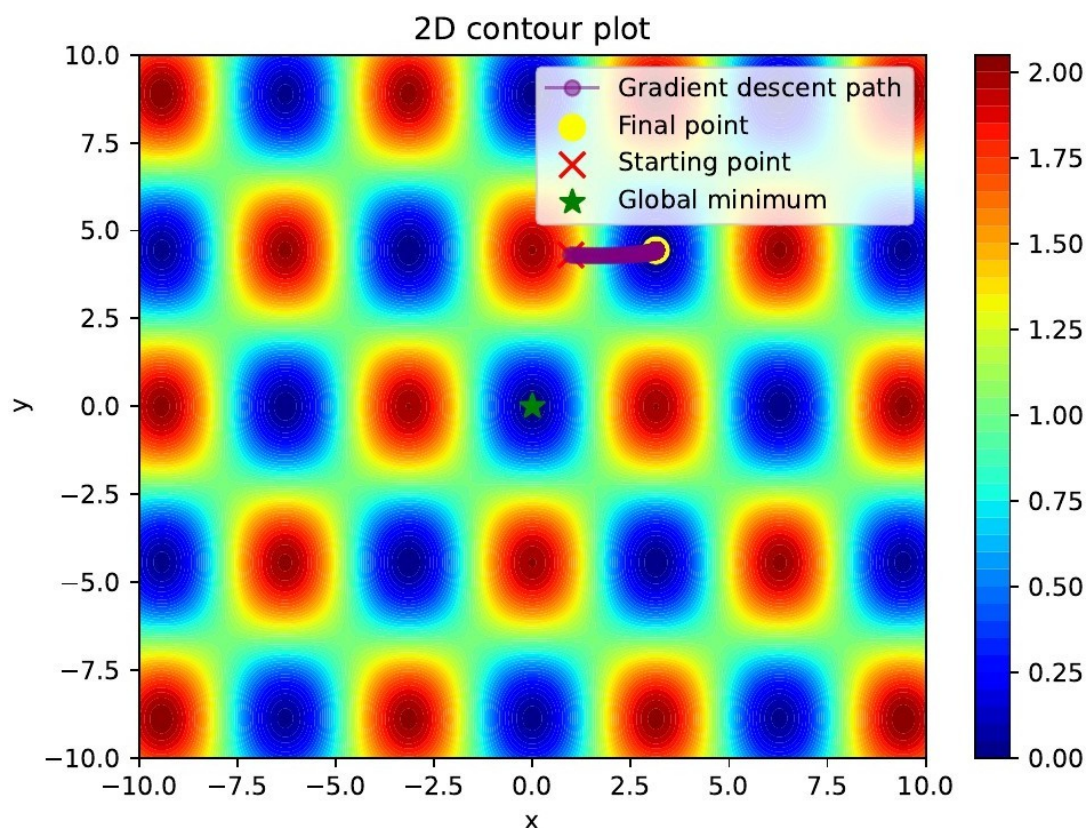


Figure 16. Contour plot with the gradient descent trajectory overlaid.

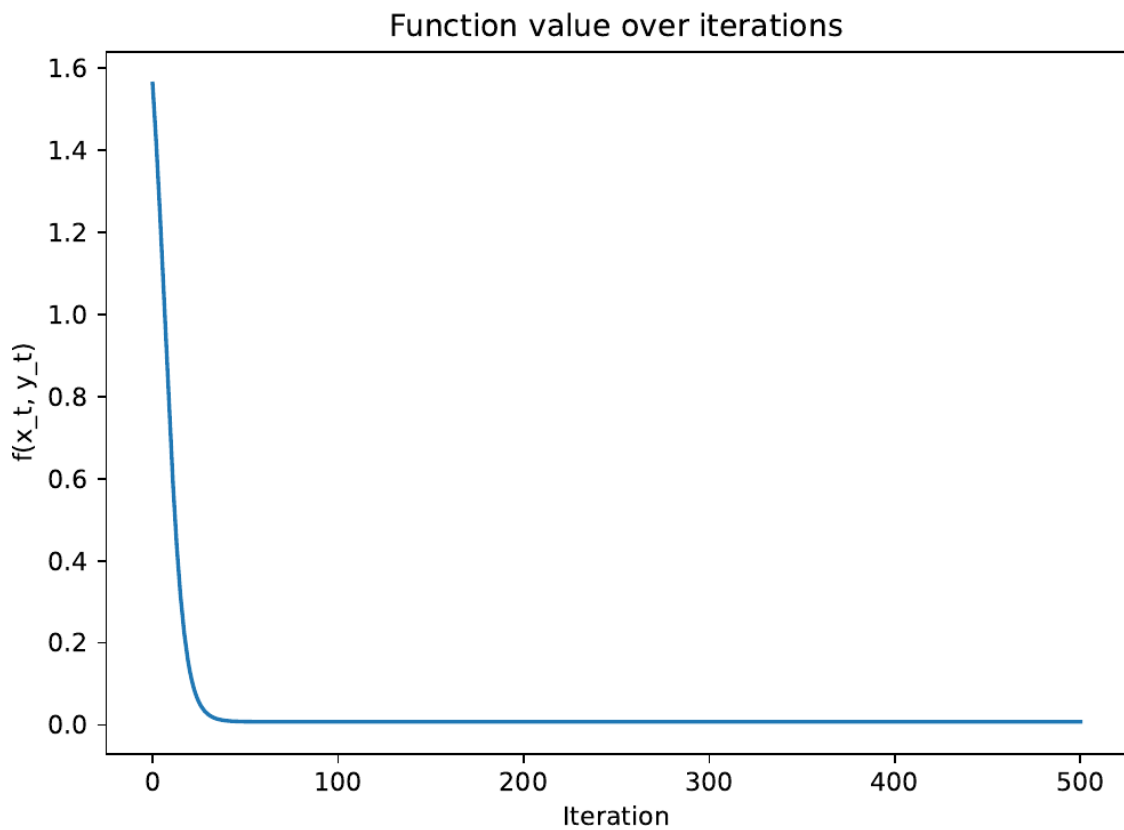


Figure 17. Objective value over gradient descent iterations.

Overall discussion

Overall, the submitted results are coherent across the three parts of the assignment. The linear-regression section shows how model quality depends on whether the data supports a linear trend; the logistic-regression section shows that carefully chosen feature maps can produce very high classification performance; and the gradient-descent section confirms both correct implementation and sensible optimization behaviour. This consistency between formulas, numerical outputs, and plots strengthens the credibility of the final submission.